

FINANCE MANAGEMENT SYSTEM REST API

Detailed Project Documentation

Java • Spring Boot • Spring Data JPA • Hibernate • MySQL • REST API

1. Project Overview

The Finance Management System REST API is a backend application designed to provide a centralized RESTful interface for managing core finance operations. The system manages customers, accounts, financial transactions, loans, and investments. Customers can maintain their personal and financial profiles; accounts are linked to customers for balance and transaction management; transactions record deposits, withdrawals, and transfers; loans maintain loan terms and repayment information; and investments maintain investment, maturity, and performance information.

The application will be developed using Java and Spring Boot and will follow a layered architecture with Controller, Service, Repository, and Entity layers. Spring Data JPA and Hibernate will be used for object-relational mapping and database operations, with MySQL as the relational database. Postman will be used to test the REST APIs.

2. Business Context

The system represents a finance management backend in which a customer may own multiple accounts. Each account can have multiple financial transactions. A customer can also have multiple loans and multiple investments. The API provides separate CRUD operations for each major finance resource.

Core business areas:

- Customer registration and profile management.
- Account creation and customer-account association.
- Account balance and transaction management.
- Deposits, withdrawals, and transfers.
- Loan management with loan terms and repayment tracking.
- Investment management with maturity and performance tracking.

3. Project Objectives

- Manage customer information.
- Create and manage customer accounts.
- Record financial transactions.
- Support deposit, withdrawal, and transfer transaction types.
- Maintain loan details and repayment tracking.
- Track customer investments.
- Maintain investment maturity and performance information.
- Provide REST APIs for CRUD operations.
- Maintain entity relationships using Spring Data JPA and Hibernate.

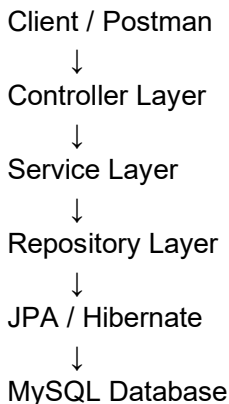
- Validate incoming API data.
- Provide centralized exception handling.
- Integrate the application with MySQL.
- Test APIs using Postman.
- Provide a clean and maintainable layered backend architecture.

4. Technology Stack

Technology	Purpose
Java	Primary programming language
Spring Boot	Backend application framework
Spring MVC	REST controller and HTTP request handling
Spring Data JPA	Repository abstraction and database operations
Hibernate	ORM and entity relationship management
MySQL	Relational database
Maven	Project and dependency management
Lombok	Reduce repetitive Java code
Jakarta Validation	Validate API request data
Postman	REST API testing
Swagger / OpenAPI	API documentation
JUnit / Mockito	Unit and service-layer testing
Spring Security / JWT	Authentication and authorization

5. Application Architecture

The application follows a layered architecture. Each layer has a specific responsibility, which keeps the application organized and makes the business logic easier to maintain and test.



Layer	Responsibility
Controller	Receives HTTP requests, validates request input, calls services, and returns HTTP responses.
Service	Contains business logic and coordinates operations between controllers and repositories.
Repository	Provides database access using Spring Data JPA.
Entity	Represents database tables and JPA relationships.
DTO	Defines request and response data exposed by the API.
Exception	Centralizes application and validation error handling.

Security	Handles authentication and authorization when security is enabled.
Config	Contains application-level configuration.

6. Main Modules

Module	Description	Key Operations
Customer	Manages customer personal and financial profile information.	Create, view, update, delete
Account	Manages accounts linked to customers and account information.	Create, view, update, delete
Transaction	Records financial activity for accounts.	Deposit, withdrawal, transfer, CRUD
Loan	Maintains loans issued to customers, terms, and repayment tracking.	Create, view, update, delete
Investment	Maintains customer investments, maturity, and performance information.	Create, view, update, delete

7. Business Flow

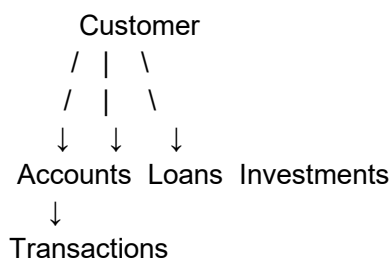
Customer Registration → Account Creation → Account Transactions → Loan Management → Investment Management

A customer is registered first. One or more accounts can then be associated with the customer. Transactions are recorded against accounts. Loans and investments are independently associated with the same customer. This structure allows the system to maintain a customer's major finance records in one backend.

8. Entity Relationships

Parent Entity	Relationship	Child Entity	Meaning
Customer	One-to-Many	Account	One customer can have multiple accounts.
Account	One-to-Many	Transaction	One account can have multiple transactions.
Customer	One-to-Many	Loan	One customer can have multiple loans.
Customer	One-to-Many	Investment	One customer can have multiple investments.

Relationship diagram:



9. Database Design

Table / Entity	Fields	Relationship / Notes
Customer	customerId, customerName, email, phone, address	Root customer record
Account	accountId, accountNumber, accountType, balance, customerId	customerId references Customer
Transaction	transactionId, transactionType, amount, transactionDate, accountId	accountId references Account
Loan	loanId, loanType, loanAmount, interestRate, duration, customerId	customerId references Customer
Investment	investmentId, investmentType, amount, maturityDate, customerId	customerId references Customer

10. Customer APIs

Method	Endpoint	Description
POST	/api/customers	Create new customer
GET	/api/customers	List all customers
GET	/api/customers/{id}	Get customer by ID
PUT	/api/customers/{id}	Update customer
DELETE	/api/customers/{id}	Delete customer

11. Account APIs

Method	Endpoint	Description
POST	/api/accounts	Create new account
GET	/api/accounts	List all accounts
GET	/api/accounts/{id}	Get account by ID
PUT	/api/accounts/{id}	Update account
DELETE	/api/accounts/{id}	Delete account

12. Transaction APIs

Method	Endpoint	Description
POST	/api/transactions	Record new transaction
GET	/api/transactions	List all transactions
GET	/api/transactions/{id}	Get transaction by ID
PUT	/api/transactions/{id}	Update transaction
DELETE	/api/transactions/{id}	Delete transaction

Transaction types supported by the finance requirements are deposit, withdrawal, and transfer.

13. Loan APIs

Method	Endpoint	Description
POST	/api/loans	Create new loan
GET	/api/loans	List all loans
GET	/api/loans/{id}	Get loan by ID
PUT	/api/loans/{id}	Update loan

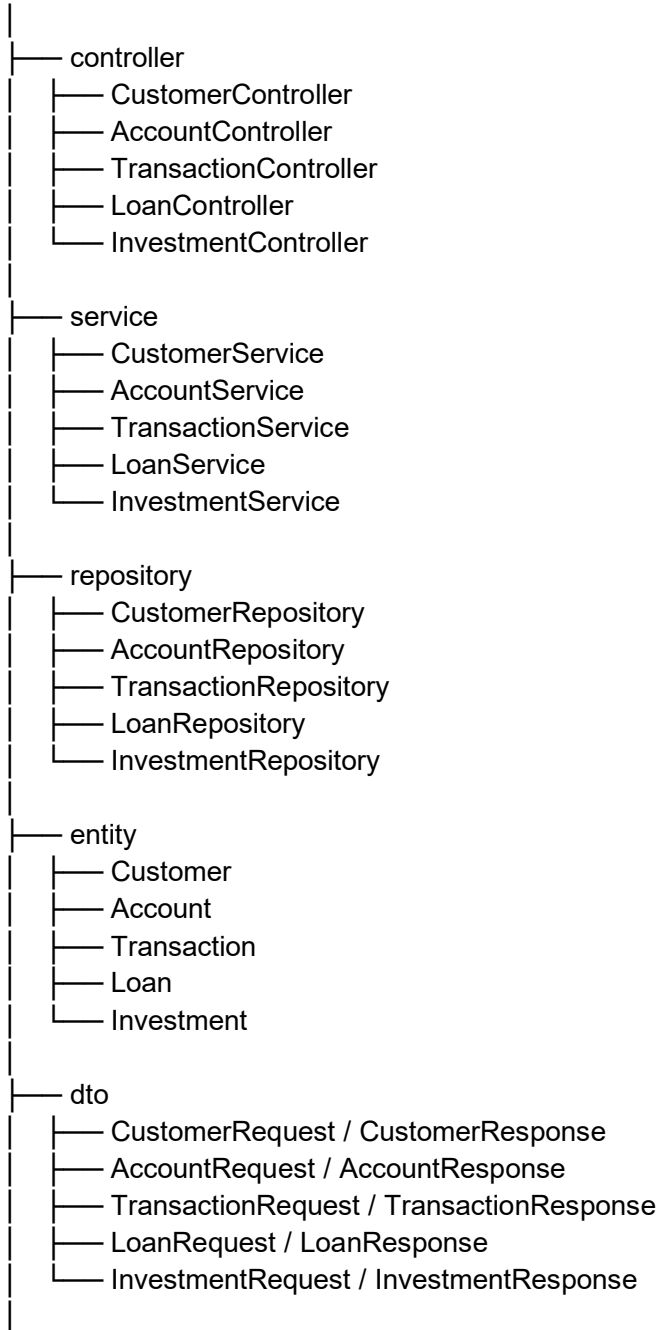
DELETE	/api/loans/{id}	Delete loan
--------	-----------------	-------------

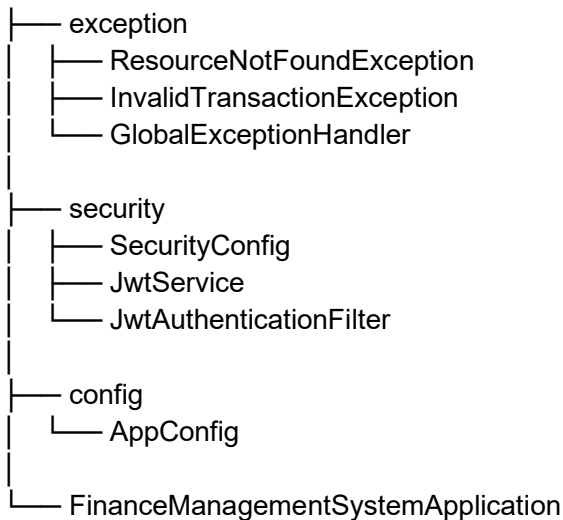
14. Investment APIs

Method	Endpoint	Description
POST	/api/investments	Create new investment
GET	/api/investments	List all investments
GET	/api/investments/{id}	Get investment by ID
PUT	/api/investments/{id}	Update investment
DELETE	/api/investments/{id}	Delete investment

15. Proposed Package Structure

FinanceManagementSystem





16. Validation

The API will validate incoming request data before processing it.

- @NotBlank – required text values must not be empty.
- @Email – validates email address format.
- @NotNull – ensures required values are provided.
- @Min / @Positive – validates numeric values where applicable.
- @Valid – triggers validation of request DTOs.

17. Exception Handling

The application will use global exception handling through @RestControllerAdvice.

ResourceNotFoundException will be used when a requested customer, account, transaction, loan, or investment does not exist. Validation and unexpected application errors will be converted into appropriate HTTP responses.

Error	HTTP Status
Invalid input / validation failure	400 Bad Request
Requested resource not found	404 Not Found
Unexpected application/server error	500 Internal Server Error

18. Transaction Management

Financial operations must preserve database consistency. The service layer will use transaction management for operations that require multiple related database changes to succeed or fail together. This is especially important for account-related financial operations.

- Use @Transactional for appropriate multi-step financial operations.
- Validate transaction type and amount before changing account data.
- Keep financial transaction records consistent with account information.
- Reject invalid financial operations using business rules and exceptions.

19. DTO Pattern

DTOs will be used to separate API request/response models from JPA entities. This keeps the database model independent from the public REST API and provides better control over validation and response data.

20. Lombok

Lombok will be used where appropriate to reduce repetitive code such as getters, setters, constructors, and other boilerplate methods in entities and DTOs.

21. Spring Core and Dependency Injection

Spring's dependency injection will be used to connect controllers, services, repositories, and supporting components. Spring-managed beans will be injected instead of manually creating dependent objects.

22. Spring AOP and Logging

Spring AOP can be used for cross-cutting concerns such as logging important service operations. Application logs will help identify requests, business operations, errors, and important finance events.

23. Swagger / OpenAPI

Swagger/OpenAPI documentation will be added so that available REST endpoints, request models, response models, and API operations can be viewed and tested through an API documentation interface.

24. HikariCP

HikariCP will be used as the database connection pool through Spring Boot's database configuration. It helps manage reusable database connections efficiently.

25. Scheduler

Scheduled processing will be used only for finance operations that require periodic checks, such as investment maturity or other time-based processing defined during implementation.

26. Actuator

Spring Boot Actuator will provide application health and monitoring information. It will be configured appropriately for the project environment.

27. Profiles

Spring profiles will allow environment-specific configuration, such as development, testing, and production database or application settings.

28. Security and JWT

Spring Security and JWT authentication will be incorporated to protect the REST APIs. Authentication and authorization rules will be defined according to the final project security requirements.

29. Caching

Caching will be considered for suitable read-heavy data where repeated database access can be reduced without causing stale or incorrect financial information. Financial transaction data will be handled carefully.

30. Testing

JUnit and Mockito will be used to test service-layer business logic and important application behavior. Testing will include successful operations and failure scenarios.

- Customer service tests.
- Account service tests.
- Transaction validation and business-rule tests.
- Loan service tests.
- Investment service tests.
- Exception and validation scenarios.

31. Postman API Testing

Postman will be used as the primary API testing client. Each module will be tested using POST, GET, PUT, and DELETE requests. Tests will also cover invalid input and missing resources.

32. HTTP Status Codes

Status Code	Meaning	Typical Usage
200 OK	Request succeeded	Successful GET or PUT operation
201 Created	Resource created	Successful POST operation
204 No Content	Request succeeded without body	Successful DELETE operation
400 Bad Request	Invalid input	Validation or invalid request
404 Not Found	Resource not found	Customer/account/transaction/loan/investment does not exist
500 Internal Server Error	Server error	Unexpected application error

33. Core Features

- Customer Management
- Account Management
- Transaction Management
- Deposit Transactions
- Withdrawal Transactions
- Transfer Transactions
- Loan Management
- Loan Repayment Tracking
- Investment Management

- Investment Maturity Tracking
- Investment Performance Tracking
- CRUD Operations
- RESTful API Design
- JPA/Hibernate Entity Relationships
- MySQL Database Integration
- Input Validation
- Global Exception Handling
- DTO-based API design
- Transaction Management
- API Documentation
- Logging
- Security
- Testing
- Monitoring and scheduled processing where applicable

34. Expected Development Flow

1. Create the Spring Boot project using Maven.
2. Configure application properties and MySQL.
3. Create Customer, Account, Transaction, Loan, and Investment entities.
4. Define JPA relationships.
5. Create repository interfaces.
6. Implement service-layer business logic.
7. Create request and response DTOs.
8. Add Jakarta Validation.
9. Implement REST controllers.
10. Add global exception handling.
11. Implement transaction-related business rules.
12. Add Swagger/OpenAPI documentation.
13. Add logging and AOP where appropriate.
14. Configure HikariCP and profiles.
15. Add Actuator and scheduled processing where applicable.
16. Implement Spring Security and JWT.
17. Add caching where appropriate.
18. Create JUnit and Mockito tests.
19. Test all APIs through Postman.
20. Prepare final project documentation and testing evidence.

35. Future Enhancements

- Advanced account balance calculation.
- Fund transfer between accounts.
- Transaction history filters.
- Advanced investment performance calculations.
- Role-Based Access Control.
- Advanced reporting and analytics.
- Docker deployment.

- API documentation portal.
- Integration with external financial or payment services.

36. Conclusion

The Finance Management System REST API is a complete finance-domain backend project covering customers, accounts, transactions, loans, and investments. The project follows the finance requirements and entity relationships defined in the provided topic material while using a clean Spring Boot layered architecture. The implementation demonstrates REST API development, JPA/Hibernate relationships, MySQL integration, validation, exception handling, transaction management, DTOs, security, testing, documentation, logging, monitoring, and other Spring features where applicable.

37. Scope Confirmation

The five core modules—Customer, Account, Transaction, Loan, and Investment—are mandatory parts of this project. The CRUD API structure and finance relationships will remain aligned with the provided project topic. Additional Spring features will enhance the implementation without removing or replacing the required finance modules.

38. Project Summary

The Finance Management System is a Spring Boot based REST API designed to manage important finance-related operations through a structured backend application. The project uses Java, Spring Boot, Spring Data JPA, Hibernate and MySQL.

The system provides customer and account management, financial transaction processing, loan management, loan document file handling and investment management. JWT-based authentication is used to secure protected APIs. The application follows a layered architecture consisting of controllers, DTOs, services, repositories and entities.

The implemented APIs were tested using Postman and the application was verified against the MySQL database. Unit testing was also performed using JUnit and Mockito. The final Maven test execution completed with 9 tests passed, 0 failures, 0 errors and 0 skipped.

Overall, this project provided practical experience in building, testing and verifying a complete backend REST API and helped connect Java and Spring Boot concepts with a real-world finance domain.

39. Learning Outcomes

During the development of the Finance Management System, I gained practical experience in developing a backend REST API using Java and Spring Boot. The project helped me understand how different Spring technologies work together in a real application.

- Java programming and object-oriented programming concepts used in backend development.
- Spring Core concepts such as dependency injection and beans.
- Spring Boot project creation, configuration and application startup.
- Spring MVC and REST API development using controllers and HTTP methods.

- DTOs for handling API request and response data.
- Jakarta Bean Validation for validating API input.
- Service-layer business logic and layered application architecture.
- Spring Data JPA and Hibernate for database persistence.
- Entity relationships such as customer-linked accounts, loans and investments.
- MySQL database design, SQL queries and database verification.
- Transaction processing, including deposits and withdrawals.
- Loan business rules such as approval, repayment and outstanding amount calculation.
- File handling using multipart upload for loan-related documents.
- JWT authentication and Spring Security for protecting APIs.
- Unit testing with JUnit and Mockito.
- API testing and verification using Postman.
- Debugging application, database and schema-related issues.